📝 Trabajo Práctico — Unidad 07

Análisis de Mediciones de Tensión de Línea

Cátedra: Programación en Computación — UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván **Tema:** Arrays unidimensionales (vectores)

🎯 Objetivos

- Declarar, inicializar y cargar vectores mediante bucles.
- Recorrer vectores con `for` y con `while`, eligiendo el adecuado en cada caso.
- Aplicar los algoritmos fundamentales: acumular, contar, buscar máximo/mínimo, búsqueda lineal.
- Escribir funciones con parámetros y valor de retorno que operen sobre vectores.
- Usar `.slice()` para no corromper los datos de entrada.
- Presentar resultados de forma clara y legible por consola.

🧩 Consignas generales

1. **Todo el código debe estar modularizado en funciones.** Nada de resolver en el cuerpo principal del programa. Cada cálculo va en su propia función, con parámetros y `return`.
2. **Cada función lleva su comentario de cabecera JSDoc:**

   ```
   /**
    * Calcula la total de todos los elementos de un vector.
    * @param {number[]} vector - Vector de números enteros
    * @returns {number} Suma total de los elementos
    */
   ```

3. **Las funciones retornan un único valor:** un número, un booleano, un string o un vector. **No se permite retornar objetos ni JSON.** Si necesitás devolver dos datos, devolvé un vector de dos elementos: `return [max, position]`.
4. **Los vectores se inicializan con bucles.** No uses `fill()` ni `push()`.
5. **Usá `.slice()`** cuando una función retorne un vector o reciba uno que no debe modificar.
6. Nombres de variables y funciones **descriptivos, en inglés o español pero consistentes**.

📘 Contexto del problema

El tablero principal de un taller registra la **tensión de línea** una vez por día. La red es monofásica de **220 V nominales**, y la norma admite una variación de **±10 %** respecto de ese valor nominal.

Toda medición que caiga fuera del rango admisible se considera una **anomalía** y debe ser reportada, porque puede dañar los motores y el equipamiento electrónico del taller.

Vas a escribir un programa que analice el registro de mediciones y produzca un informe.

> 💡 Definí la tensión nominal y la tolerancia como **constantes** al principio del programa:
>
> ```
> const NOMINAL_VOLTAGE = 220
> const TOLERANCE_PERCENT = 10
> ```
>
> Así, si mañana el taller pasa a 380 V, cambiás una línea y no veinte.

Problemas

Problema 1 — Generación del registro de mediciones

Escribí una función que genere un vector de mediciones de tensión, con valores **enteros aleatorios** dentro de un rango que se recibe por parámetro.

```
const getMeasurements = (size, min, max) => { ... }
```

Reusá la función `rndInt()` de la unidad 06.

El programa debe pedir por consola la **cantidad de mediciones** y el **rango** de valores, y mostrar el vector generado con `console.table()`.

Ejemplo de ejecución:

```
Ingrese la cantidad de mediciones: 8
Ingrese la tensión mínima posible [V]: 195
Ingrese la tensión máxima posible [V]: 245

Mediciones registradas [V]:
┌─────────┬────────┐
│ (index) │ Values │
├─────────┼────────┤
│ 0       │ 219    │
│ 1       │ 235    │
│ 2       │ 210    │
│ 3       │ 195    │
│ 4       │ 221    │
│ 5       │ 245    │
│ 6       │ 220    │
│ 7       │ 205    │
└─────────┴────────┘
```

> Los valores son aleatorios, así que tu salida va a mostrar otros números. **Para poder comparar tus resultados con los de este enunciado**, mientras desarrollás podés reemplazar la llamada a `getMeasurements()` por el vector fijo del ejemplo:
>
> ```
> const measurements = [219, 235, 210, 195, 221, 245, 220, 205]
> ```
>
> Todos los ejemplos de salida que siguen usan ese vector.

Problema 2 — Tensión media

Escribí las funciones necesarias para calcular la **tensión media** del período registrado: la suma de todas las mediciones dividida la cantidad.

```
media = (medición[0] + medición[1] + ... + medición[n-1]) / n
```

```
const sumAll = (vector) => { ... }
const getAverage = (vector) => { ... }
```

Mostrá el resultado con **dos decimales** (investigá `.toFixed(2)`).

Ejemplo de salida esperada:

```
--- Problema 2: Promedio ---
Tensión media: 218.75 V
```

Problema 3 — Tensión máxima y mínima

Escribí dos funciones que determinen la tensión **máxima** y la **mínima** del registro, **junto con el número de medición** en que se produjo cada una.

```
const findMax = (vector) => { ... }   // retorna [valor, position]
const findMin = (vector) => { ... }   // retorna [valor, position]
```

Ejemplo de salida esperada:

```
--- Problema 3: Maximo y minimo ---
Tensión máxima: 245 V (medición Nro 5)
Tensión mínima: 195 V (medición Nro 3)
```

> ⚠️ **Inicializá el máximo y el mínimo con `vector[0]`, nunca con `0`.** Si inicializás en `0`, el mínimo va a dar siempre `0` — un valor que ni siquiera está en el registro. Esto se corrige como **error grave**.

Problema 4 — Mediciones fuera de rango

Escribí una función que determine si **una** tensión está dentro del rango admisible, y otra que **cuente** cuántas mediciones del registro quedaron fuera de ese rango. El rango admisible va de la tensión nominal menos la tolerancia a la nominal más la tolerancia:

```
límite inferior = nominal - nominal * tolerance / 100
límite superior = nominal + nominal * tolerance / 100

dentro de rango   si   límite inferior <= tensión <= límite superior
porcentaje        =    fuera de rango / cantidad de mediciones * 100
```

```
const isWithinRange = (voltage, nominal, tolerance) => { ... }   // retorna boolean
const countOutOfRange = (measurements, nominal, tolerance) => { ... }
```

Mostrá también el **porcentaje** de mediciones anómalas respecto del total.

Ejemplo de salida esperada:

```
Tensión nominal:   220 V
Tolerancia:        +/-10%
Rango admisible:   198 V a 242 V

--- Problema 4: Mediciones fuera de rango ---
Mediciones fuera de rango: 2 de 8 (25.00%)
```

> ⚠️ **Calculá los límites como dice la fórmula.** La forma «equivalente» `nominal * (1 + tolerance / 100)` da `242.00000000000003` en vez de `242`: es el error de punto flotante de la unidad 03.

> 💡 Fijate que `isWithinRange()` es una función chiquita que devuelve `true` o `false`, y `countOutOfRange()` la usa adentro de su bucle. Eso es **componer funciones**: cada una hace una sola cosa. Vas a reusar `isWithinRange()` en los problemas 5 y 7.

Problema 5 — Primera anomalía

Escribí una función que devuelva el **número de la primera medición** que quedó fuera de rango. Si todas las mediciones están dentro de rango, debe devolver `-1`.

```
const firstOutOfRange = (measurements, nominal, tolerance) => { ... }
```

El programa debe interpretar ese `-1` e imprimir un mensaje claro en cada caso.

Ejemplo de salida esperada (caso con anomalías):

```
--- Problema 5: Primera anomalia ---
Primera medición fuera de rango: Nro 3 (195 V)
```

Ejemplo de salida esperada (caso sin anomalías):

```
--- Problema 5: Primera anomalia ---
No se registraron mediciones fuera de rango
```

> 💡 Acá conviene un `while`, no un `for`: en cuanto encontrás la primera anomalía no tiene sentido seguir recorriendo el resto del vector.

Problema 6 — Desviación porcentual

Escribí una función que, a partir del vector de mediciones, devuelva un **vector nuevo** con la **desviación porcentual** de cada medición respecto de la tensión nominal:

$$\text{desviación}[i] = \frac{\text{medición}[i] - \text{nominal}}{\text{nominal}} \times 100$$

```
const getDeviations = (measurements, nominal) => { ... }
```

La función no debe modificar el vector de mediciones original. Después de llamarla, imprimí el vector original para demostrar que quedó intacto.

Ejemplo de salida esperada:

```
--- Problema 6: Desviaciones porcentuales ---
  Medición 0: 219 V -> -0.45%
  Medición 1: 235 V -> 6.82%
  Medición 2: 210 V -> -4.55%
  Medición 3: 195 V -> -11.36%
  Medición 4: 221 V -> 0.45%
  Medición 5: 245 V -> 11.36%
  Medición 6: 220 V -> 0.00%
  Medición 7: 205 V -> -6.82%

Vector original luego de calcular las desviaciones:
[ 219, 235, 210, 195, 221, 245, 220, 205 ]
(debe estar intacto: por eso se usa .slice())
```

> 🔬 **Experimento obligatorio.** Una vez que funcione, borrá el `.slice()` de tu función, corré el programa de nuevo y mirá qué pasa con el vector original. Escribí en un comentario **por qué** cambia. Este es el concepto más importante del TP.

Problema 7 — Racha más larga en régimen normal

Escribí una función que determine la **cantidad máxima de mediciones consecutivas** que se mantuvieron dentro del rango admisible, y **en qué medición comienza** esa racha.

```
const longestRun = (measurements, nominal, tolerance) => { ... }   // retorna [longitud, inicio
```

Ejemplo de salida esperada:

```
--- Problema 7: Racha mas larga en rango ---
Racha más larga dentro de rango: 3 mediciones consecutivas
Comienza en la medición Nro 0
```

> 💡 **Pista:** necesitás dos pares de variables: la racha *actual* (que se reinicia cuando aparece una anomalía) y la *mejor* racha encontrada hasta el momento. Antes de codificar, resolvelo a mano con el vector del ejemplo y anotá cómo van cambiando esas cuatro variables en cada paso.

🏁 El informe completo

Al final, el programa debe imprimir un informe ordenado con todos los resultados. Apuntá a una salida prolija: encabezados, unidades (`V`, `%`) y alineación.

Ejemplo de informe completo:

```
============================================================
   ANALISIS DE MEDICIONES DE TENSION DE LINEA
============================================================

Mediciones registradas [V]:
┌─────────┬────────┐
│ (index) │ Values │
├─────────┼────────┤
│ 0       │ 219    │
│ 1       │ 235    │
│ 2       │ 210    │
│ 3       │ 195    │
│ 4       │ 221    │
│ 5       │ 245    │
│ 6       │ 220    │
│ 7       │ 205    │
└─────────┴────────┘
Tensión nominal:   220 V
Tolerancia:        +/-10%
Rango admisible:   198 V a 242 V

Tensión media: 218.75 V
Tensión máxima: 245 V (medición Nro 5)
Tensión mínima: 195 V (medición Nro 3)
Mediciones fuera de rango: 2 de 8 (25.00%)
Primera medición fuera de rango: Nro 3 (195 V)
Racha más larga dentro de rango: 3 mediciones, desde la Nro 0

============================================================
                   FIN DEL ANALISIS
============================================================
```

⛔ Restricciones

Permitido:

- Bucles `for`, `while`, `do-while`
- `if` / `else if` / `else`, `switch-case`
- Acceso por índice: `vector[i]`
- `.length` y `.slice()`
- `Math.random()`, `Math.floor()`, `.toFixed()`
- `console.log()` y `console.table()`

No permitido:

- Métodos de array: `map`, `filter`, `reduce`, `forEach`, `find`, `sort`, `flat`, `indexOf`, `splice`

- `fill()` y `push()` para inicializar vectores
- Retornar objetos o JSON desde una función
- Cualquier método o técnica no vista en clase

📎 Material de consulta

- Apunte de la unidad (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/apunte.pdf)
- Presentación de clase (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/presentacion.pdf)
- Ejemplos desarrollados en clase (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/ejemplos)